Ministerul Educaţiei şi Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică şi Microelectronică
Departamentul Ingineria Software şi Automatică
Program: Software Engineering

Report on laboratory work no. 3

Course: Operating Systems

Topic: The Command Line: Shell, Pipes and Text Filters

Track A: the real OS, from the outside (Linux)

Done by:
st. gr. FAF-241,
Serghei Barladean

Checked by:
Patricia Reitman

Chişinău – 2026

Contents

Purpose of the Work **3**
Part 1. Move Fast, Find Help **3**
1.1. Work Done 3
1.2. Observations 4
Part 2. What the Shell Does to Your Line **5**
2.1. Work Done 5
2.2. Predictions and Real Output 5
2.3. Observations 6
Part 3. Redirection, Chaining and Jobs **7**
3.1. Work Done 7
3.2. Observations 8
Part 4. Answering Questions with One-Liners **9**
4.1. Data Set 9
4.2. Questions Q1–Q8 9
4.3. Full Run, Checks and Bonus 11
Part 5. Your First Script **12**
5.1. Work Done 12
5.2. Observations 13
Conclusions **13**

Purpose of the Work

The purpose of this laboratory work is to move fast in the bash command line, to predict what the shell does to a command line before it runs (globbing, quoting and expansions), to redirect the output of commands, to chain commands together, and to answer questions about data with one-line pipelines built from small text filters. At the end a first shell script is written.

The work was done in bash on Ubuntu 24.04 (aarch64) running in a Multipass virtual machine on macOS. Every command below was actually run in this virtual machine; the figures are taken from the recorded terminal session.

Part 1. Move Fast, Find Help

1.1. Work Done

The command `ls /usr/sh` followed by Tab was completed by the shell to `ls /usr/share/`, and `cd /et` followed by Tab was completed to `cd /etc/`. In both cases there was only one matching name, so bash completed it immediately instead of showing a list of candidates.

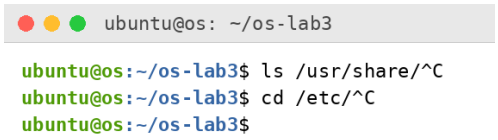


Figure 1.1 – Tab completion

The kind of each command and the way the shell finds it were checked with `type`, `which` and the `PATH` variable:

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog ; echo "exit code: $?"
```

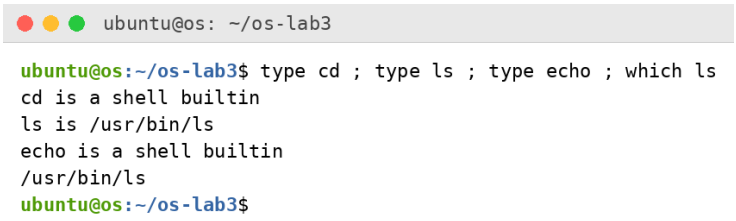


Figure 1.2 – Output of type and which

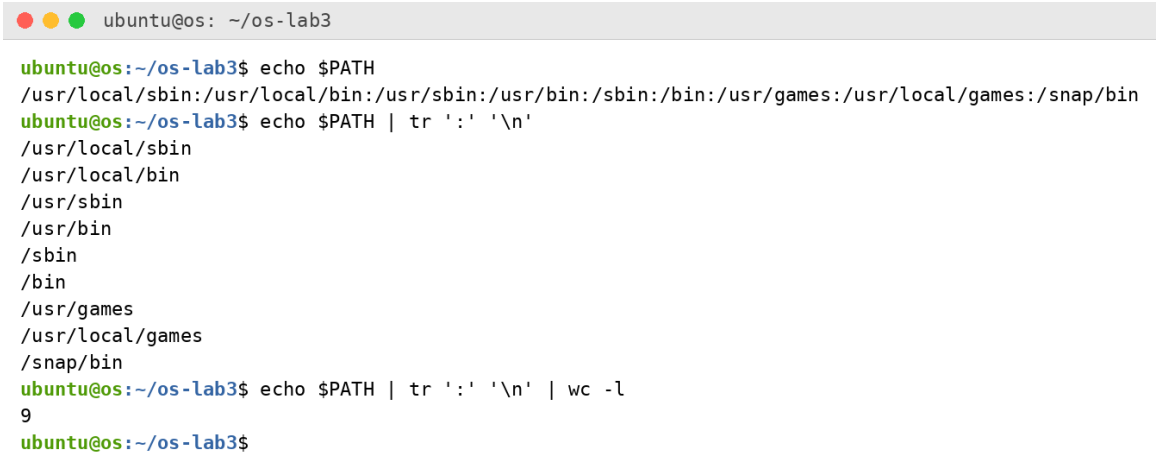


Figure 1.3 – The directories in PATH

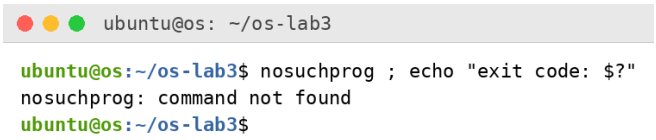


Figure 1.4 – Exit code of a missing command

Help was found without a browser with ls --help, help cd and man -k:

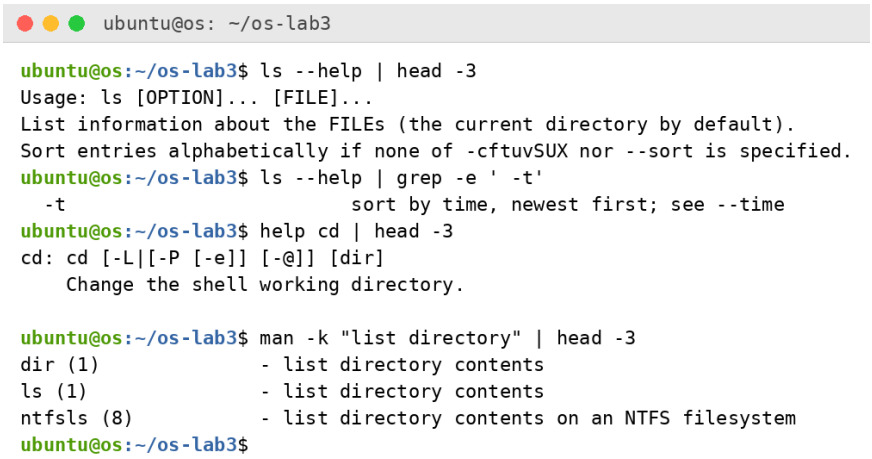


Figure 1.5 – Help from the terminal: ls --help, help cd and man -k

1.2. Observations

1. `cd` and `echo` are shell built-ins ("cd is a shell builtin"), while `ls` is an external program, `/usr/bin/ls`. `cd` must be a built-in because the current working directory is a property of the process. If cd were a separate program, the shell would start it as a child process, the child would change its own directory and exit, and the shell would stay where it was.
2. There are 9 directories in PATH: /usr/local/sbin, /usr/local/bin, /usr/sbin, /usr/bin, /sbin, /bin, /usr/games, /usr/local/games and /snap/bin. The shell searches them from left to right and `ls` is found in `/usr/bin`.
3. A command that is not found returns exit code 127 (the shell printed "nosuchprog: command not found" and `$?` was 127).

4. The option `-t` of ls sorts by modification time, newest first ("-t sort by time, newest first" in ls --help).

Part 2. What the Shell Does to Your Line

2.1. Work Done

A test directory with a few files was created:

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

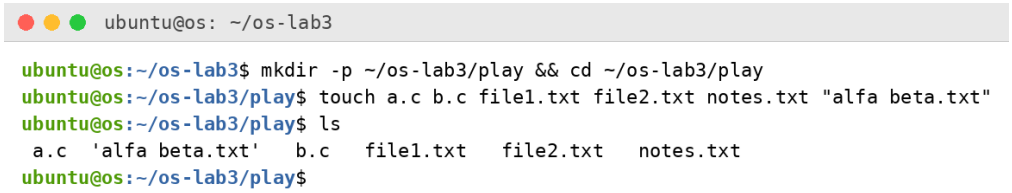


Figure 2.1 – Files created in the play directory

Note that ls prints the file with a space in quotes, 'alfa beta.txt'. GNU ls does this when its output goes to a terminal: it quotes names that contain special characters such as a space, so that the name can be copied back into a command safely. When the output goes into a pipe or a file, the quotes are not added.

2.2. Predictions and Real Output

For every line the expected output is compared with the real output of the command:

Command	My prediction	Real output	Explanation
`echo *.c`	a.c b.c	a.c b.c	The shell expands *.c to the matching file names before echo starts.
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt	? matches exactly one character.
`echo [a-b]*`	a.c alfa beta.txt b.c	a.c alfa beta.txt b.c	[a-b] matches one character, a or b, and * matches the rest. The name with the space is one word for the shell.
`echo *.xyz`	*.xyz	*.xyz	Nothing matches, so bash leaves the pattern unchanged and echo prints it.
`ls alfa beta.txt`	error: two names not found	ls: cannot access 'alfa': No such file or directory ls: cannot access 'beta.txt': No such file or directory	The space splits the line into two arguments, alfa and beta.txt.
`ls "alfa beta.txt"`	'alfa beta.txt'	'alfa beta.txt'	Double quotes keep the name as one argument.
`echo '$HOME' "$HOME" \$HOME`	$HOME /home/ubuntu $HOME	$HOME /home/ubuntu $HOME	Single quotes and the backslash keep $HOME literal; in double quotes the variable is expanded.
`echo {1..5} ; echo $((7 * 6))`	1 2 3 4 5 and 42	1 2 3 4 5 42	Brace expansion generates the words; $(( )) is arithmetic expansion.
`echo "today is $(date +%A), I am $(whoami)"`	today is Sunday, I am ubuntu	today is Sunday, I am ubuntu	$( ) is command substitution: the output of the inner command replaces it.

Command	My prediction	Real output	Explanation
`touch log{1..3}.txt && ls log*`	log1.txt log2.txt log3.txt	log1.txt log2.txt log3.txt	Brace expansion creates three names; && runs ls only if touch succeeded; log* is a glob.



Figure 2.2 – Globbing results

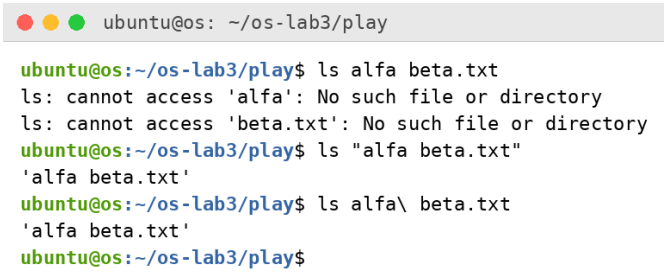


Figure 2.3 – The ls error on a name with a space, and the fix

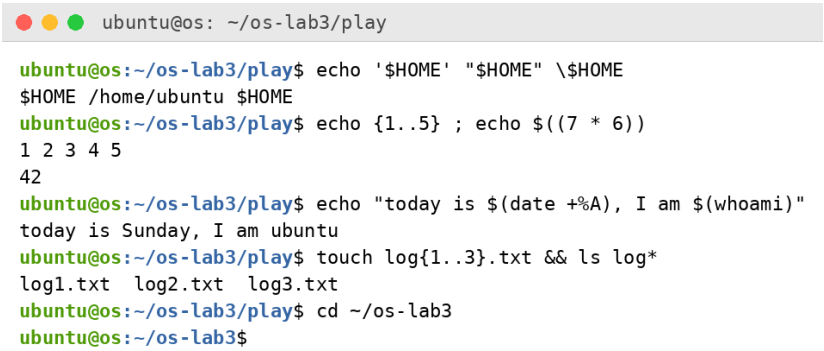


Figure 2.4 – Single quotes, double quotes and backslash, brace expansion, arithmetic and command substitution

2.3. Observations

1. The expected outputs, the real outputs and the explanations are given in the table above. The real outputs agreed with the expected ones.
2. `echo` knows nothing about files. The file names were produced by the shell (bash): it expands the pattern *.c into the list of matching names (globbing) and only then starts echo with those names as arguments.
3. `ls alfa beta.txt` failed because the space separates words, so ls received two arguments, alfa and beta.txt, and neither file exists. Two ways to fix it: put the name in quotes, `ls "alfa beta.txt"`, or escape the space with a backslash, `ls alfa\ beta.txt`. Both were run and both worked.

4. Single quotes keep everything literal: no variable, command or glob is expanded inside them ('$HOME' stays $HOME). Double quotes also keep the text as one word and stop globbing, but still allow $ expansions, so "$HOME" became /home/ubuntu. The backslash escapes only the one following character (\$HOME stays $HOME).

Part 3. Redirection, Chaining and Jobs

3.1. Work Done

Standard output (file descriptor 1) and standard error (file descriptor 2) were sent to different files:

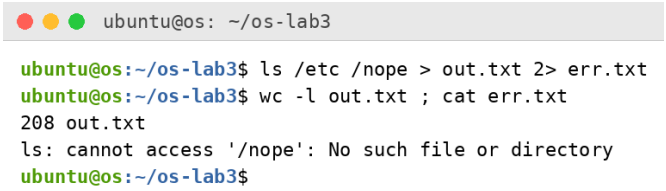


Figure 3.1 – stdout and stderr in separate files

The two orders of the redirection operators were compared. In the file all.txt the error message was counted with grep -c:

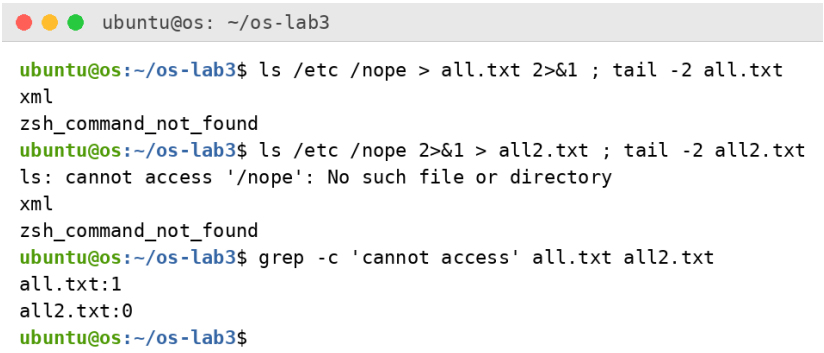


Figure 3.2 – Comparing the order of redirections

Then output redirection with > and >>, input redirection with <, the operators ;, && and ||, and command substitution were tried:

```
ubuntu@os: ~/os-lab3
ubuntu@os:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
ubuntu@os:~/os-lab3$ wc -l < access.log
500
ubuntu@os:~/os-lab3$ wc -l access.log
500 access.log
ubuntu@os:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
ubuntu@os:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/ubuntu/os-lab3/tmp
ubuntu@os:~/os-lab3$ true && echo "and: runs because the first succeeded" ; false && echo "never pri
nted" ; false || echo "or: runs because the first failed"
and: runs because the first succeeded
or: runs because the first failed
ubuntu@os:~/os-lab3$
```

Figure 3.3 – The ; && || operators and other redirections

A background job was started with & and stopped with kill:

```
ubuntu@os: ~/os-lab3
ubuntu@os:~/os-lab3$ sleep 100 &
[1] 52181
ubuntu@os:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
ubuntu@os:~/os-lab3$ jobs
[1]+  Terminated              sleep 100
ubuntu@os:~/os-lab3$
```

Figure 3.4 – Running a background job and stopping it

3.2. Observations

1. The error message ("ls: cannot access '/nope': No such file or directory") went to `err.txt` and the listing of /etc (208 lines) went to `out.txt`.
2. `> all.txt 2>&1` and `2>&1 > all.txt` differ because the shell processes redirections from left to right. In the first one stdout is sent to all.txt first, and then stderr is made a copy of stdout, so both end up in the file (grep -c found the error once in all.txt). In the second one stderr is first made a copy of the current stdout, which is still the terminal, and only then stdout is sent to all2.txt. The error therefore appeared on the screen and the file contains only the listing (grep -c found 0 errors in all2.txt).
3. `wc -l < access.log` prints only the number because wc reads from standard input and does not know any file name. With `wc -l access.log` the file name is an argument, so wc prints it after the count.
4. `;` runs the next command after the previous one finishes, whatever its result. `&&` runs the next command only if the previous one succeeded (exit code 0). `||` runs the next command only if the previous one failed (non-zero exit code).

Part 4. Answering Questions with One-Liners

4.1. Data Set

The artificial web access log was generated with the command given in the lab. Each line has the date, time, user, result (OK or FAIL) and page:

```
for i in $(seq 1 500); do echo "2026-10-05 10:$(printf %02d $((i %
60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] && echo FAIL || echo
OK) /page$((i % 4))"; done > access.log
head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

4.2. Questions Q1–Q8

Q1. How many requests failed?

```
grep -c FAIL access.log
```

100 requests failed.

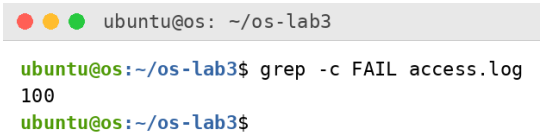


Figure 4.1 – Number of failed requests (Q1)

Q2. How many different pages were requested, and which?

```
cut -d' ' -f5 access.log | sort -u | awk '{print} END{print NR,
"different pages"}'
```

4 different pages: /page0, /page1, /page2 and /page3.

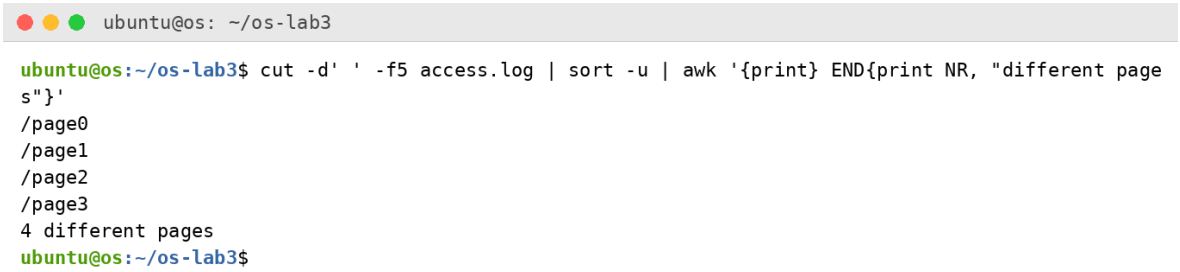


Figure 4.2 – Different pages (Q2)

Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c
```

Each of the four pages got 125 requests (4 × 125 = 500).

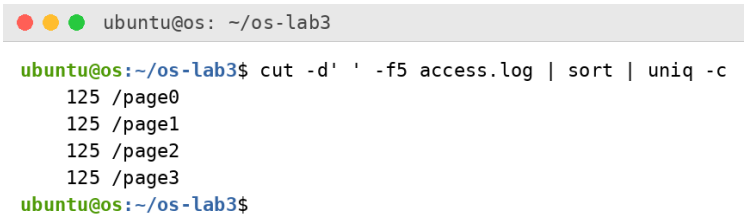


Figure 4.3 – Requests per page (Q3)

Q4. Which user or users have the most failed requests, and how many?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk
'NR==1{m=$1} $1==m'
```

user3 and user5 share the maximum, 15 failed requests each.

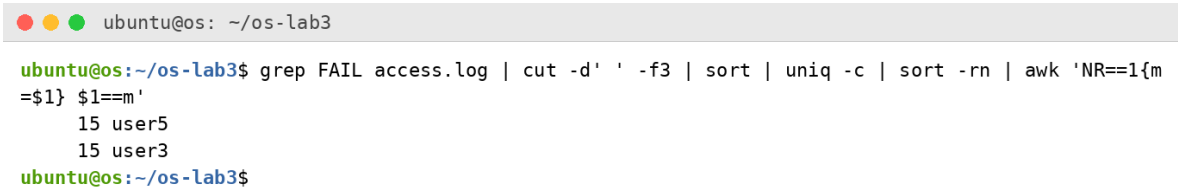


Figure 4.4 – Users with the most failed requests (Q4)

Q5. How many requests did user3 make, and how many of them failed?

```
grep -w user3 access.log | awk '{t++; if($4=="FAIL") f++} END{print t"
requests, "f" failed"}'
```

user3 made 72 requests, 15 of them failed.

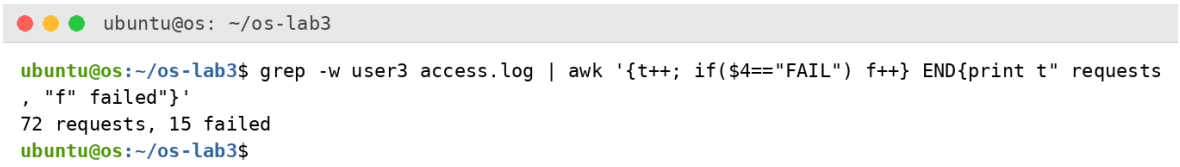


Figure 4.5 – Requests of user3 (Q5)

Q6. Print the last 3 failed requests, showing only time and user.

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

10:10 user0, 10:15 user5, 10:20 user3.

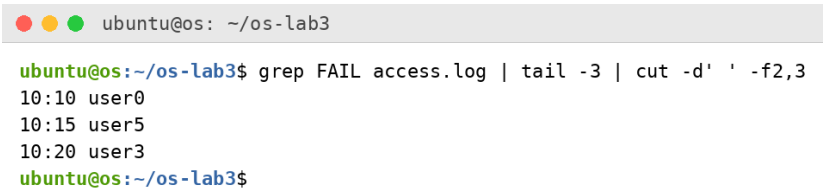


Figure 4.6 – The last three failed requests (Q6)

Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

/usr/sbin/nologin: 27 accounts, /bin/false: 2, /bin/bash: 2, /bin/sync: 1.

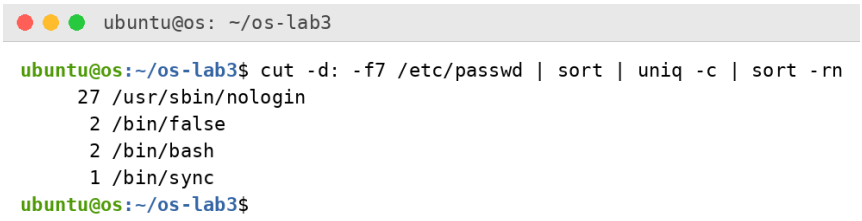


Figure 4.7 – Login shells in /etc/passwd (Q7)

Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?

```
ls /etc | wc -l ; ls -l /etc | wc -l
```

The first command printed 207 and the second 208. The long format ls -l prints one extra line at the top, "total N" (the number of disk blocks used by the listed files), before the entries, so wc counts one line more.

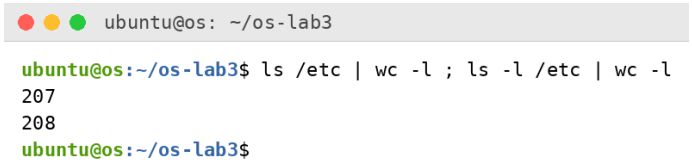


Figure 4.8 – Line counts of ls and ls -l (Q8)

4.3. Full Run, Checks and Bonus

All one-liners were saved in `oneliners.sh`, one per question with the question as a comment above it, and the file was run with `bash oneliners.sh`:

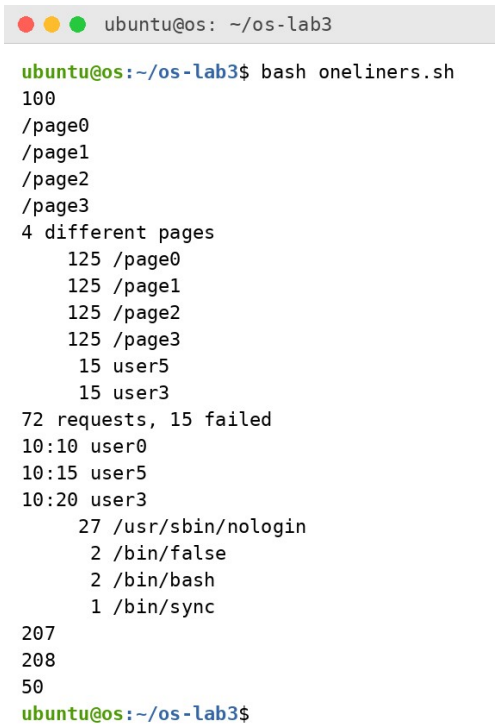


Figure 4.9 – Full run of oneliners.sh

Checks: the counts of Q3 add up to 500 (4 × 125), as the log has 500 lines. The failures per user (see Part 5: 14, 14, 14, 15, 14, 15, 14) add up to 100, which equals the answer to Q1.

Bonus: with `grep -E` the lines where the minute ends in 0 and the request failed were selected. There are 50 of them.

```
grep -E ' 10:[0-5]0 user[0-9] FAIL ' access.log | wc -l
```

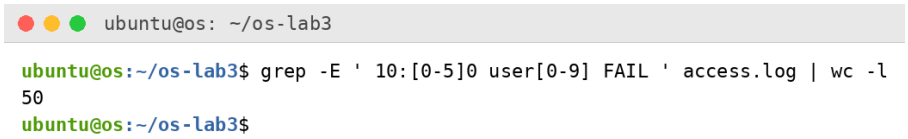


Figure 4.10 – Bonus: failed requests at a full ten minutes

Part 5. Your First Script

5.1. Work Done

The script `report.sh` was created from the template of the lab. The TODO was completed with a loop over the users user0 … user6 that counts the lines with FAIL for each user:

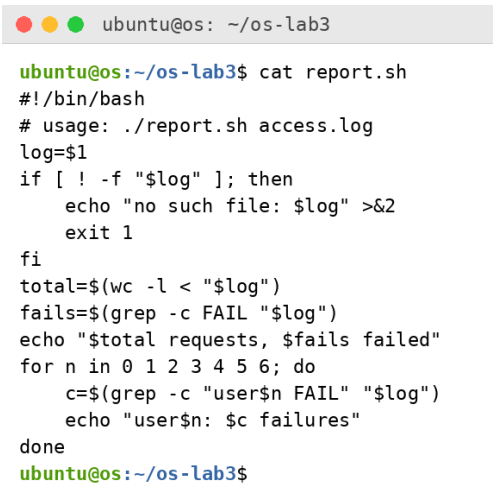


Figure 5.1 – Contents of report.sh

The script was first run before it was made executable, and without ./, then made executable with chmod +x and run on the log:

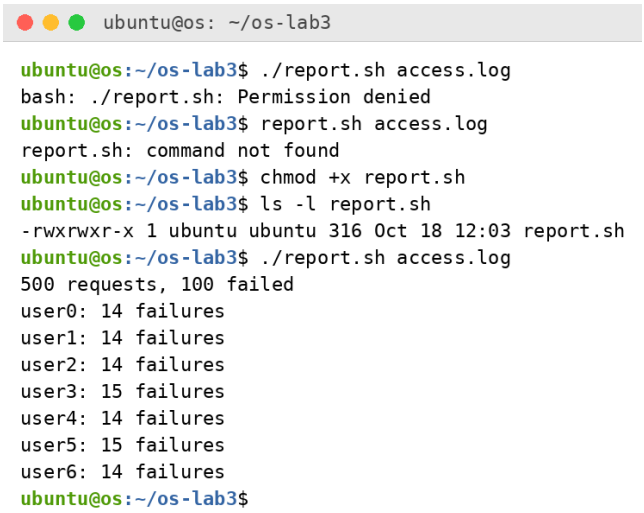


Figure 5.2 – Running the script on access.log (before and after chmod +x)

With a file that does not exist the script reports the error and returns exit code 1:

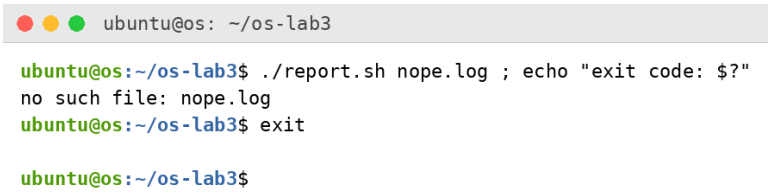


Figure 5.3 – Error handling for a missing file

5.2. Observations

1. `#!/bin/bash` (the shebang) tells the kernel which interpreter must run the file. Before `chmod +x` the run `./report.sh access.log` failed with "Permission denied", because the file did not have the execute permission.
2. The current directory is not in PATH, and the shell looks for a command name without a slash only in the PATH directories. Therefore `report.sh` gave "command not found". Writing `./report.sh` gives the path explicitly.
3. `>&2` sends the error message to standard error, so it is not mixed with the normal output of the script and can be redirected separately. `exit 1` ends the script with a non-zero exit code, which tells the caller that the script failed (the run above printed "exit code: 1"); success is exit code 0.

Of the bonus tasks, the grep -E bonus of Part 4 was done. The cron bonus was not done.

Conclusions

During this laboratory work the bash command line was explored on a real Linux system. Tab completion and the help tools make work faster; type, which and PATH show how the shell finds a command; the exit code of a missing command is 127.

The shell, not the program, expands globs, variables, braces and command substitutions and removes the quotes before the program starts, which was seen in the predictions table and in the case of the file name with a space. Standard output and standard error are separate streams, and the order of the redirections matters; the operators ;, && and || chain commands by their exit codes.

With grep, cut, sort, uniq, wc, head, tail and awk joined by pipes, all eight questions about the 500-line access log were answered with one line each, and the results were checked (125 × 4 = 500 requests, 100 failures). Finally, the first shell script report.sh was written, made executable and tested, including the error path with exit code 1.